DEEP-DIVE ARCHITECTURAL & LOGIC AUDIT #05F: NON-WEAPON EQUIPMENT & PROCEDURAL SYSTEM (PART 6 — PENULTIMATE DEEP AUDIT: PROCEDURAL EQUIPMENT ENGINE, `ITEM_PROC_*` / `REC_PROC_*` TEMPLATES, `slotOptions`, `materialMods`, 12 AFFIXES, DURABILITY & SALVAGE)

- **Target Codebase**: `JianghuSect 2` (`JS2-v2.6.0` / `SIM.PATCH_VERSION = '2.6.0'` / `HC.FINGERPRINT = '155:2f9c9dd7'`)
- **Target Audience**: Autonomous AI Software Engineers (**GPT** & **Gemini** Models)
- **Master Roadmap (`05A–05G`)**:
  - `05A–05D`: Granular Non-Weapon Group Audits (`Armors`, `Jewelry/Shoes/Pouches`, `Talismans/Cores/Tools`, `Sets/Pairs/Catalysts`)
  - `05E`: Holistic Overview of All Non-Weapon Equipment (`333` Items)
  - **`05F` (This Penultimate Document)**: **Deep Audit of the Procedural Equipment Engine (`ECO.rollProceduralGear`), All `23` `ITEM_PROC_*` / `REC_PROC_*` Templates, `slotOptions` & `materialMods`, `PROCEDURAL_POOLS` & `12` Affixes, Resonance Tagging, Durability/Repair Math & Salvage Sink**
  - `05G`: Part 7 — Grand Finale: Master End-to-End Audit of the Entire Equipment Ecosystem (`898` Items + All Connected Systems)

---

0. Hard Architectural Invariants for AI Implementers

1. **Immutable `human_core.js`**: Never modify `GAME_SOURCE/js/human_core.js` (`HC.FINGERPRINT === '155:2f9c9dd7'`).
2. **Single PRNG & Concrete Persistence Rule**: Never use `Math.random()`; always roll via `W.rng` (or explicit seeded `U.RNG` when detached) and persist concrete rolled `st`, `baseSt`, `affixes`, `nameTh`, `uid`, and `origin`—never re-roll procedural stats on save/load.

3. **Strict Tier-4 Procedural Ceiling**: Procedural equipment spans **Tier 1–4** (`Common` $\to$ `Exceptional`); Tiers 5–7 are reserved for rare-ore authored recipes, Pagoda sets, Sanctum relics, and Unique Legends.

---

1. [PROC-REC-01] The 18 / 23 Procedural Recipe Registration Gap & Why Autonomous Forge Orders Almost Never Craft Procedural Gear (economy.js:88, 97, 158, 274, 448)!

Cross-checking `equip_repro_result.log:1` (`recipes: 895, proceduralRecipes: 18` in `JD.ITEMS.recipes`) against `PATCH_IMPLEMENTATION_REPORT_JIANGHUSECT2_V2_3_0.md` (`1,370` recipes in `ECO.recipeById`) and `ALL_GEAR_TYPES` (`23` procedural types: `16 WEAPON + 3 ARMOR + 4 ACCESSORY`) exposes **three critical template & pipeline bugs**:

1.1 5 Missing `proceduralOut` / `JD.ITEMS.recipes` Registrations (18 vs 23 Templates & 475 Invisible Recipes!)

- `ECO.recipesFor(W, crafts)` (`js/economy.js:88`)—which feeds **`ECO.pickOrder` for every autonomous Sect Forge & Alchemy Hall order in the game**—iterates **`JD.ITEMS.recipes.filter(...)`**, NOT `Object.values(ECO.recipeById)`!
- **Catastrophic Consequence**:
  1. Any expansion recipe registered into `ECO.recipeById` *after* `economy.js` initialized without also pushing into `JD.ITEMS.recipes` (`1,370 - 895 = 475` recipes!) is **100% invisible to autonomous NPC production**!
  2. Specifically among the `23 REC_PROC_<TYPE>` molds, **5 procedural recipes** either lack `proceduralOut: true` (causing `ECO.finishCraft line 274` to fall into `ECO.mintGear` instead of `ECO.rollProceduralGear`, minting an unrolled dummy `ITEM_PROC_*`!) or are missing from `JD.ITEMS.recipes`!
- **Fix**: Synchronize `JD.ITEMS.recipes` $\leftrightarrow$ `ECO.recipeById` so `ECO.recipesFor` sees all `1,370` recipes, and assert in `validate_data_schema.js` that all **23** `ALL_GEAR_TYPES` have a registered `ITEM_PROC_<TYPE>` and `REC_PROC_<TYPE>` with `proceduralOut: true`, `proceduralSlot`, `proceduralGearType`, and valid `slotOptions`!

1.2 Shared Template ID (`ITEM_PROC_<TYPE>`) Tricks `ECO.pickOrder` Into Blocking Procedural Forging (economy.js:425, 448–453)!

- Look at `js/economy.js:425 & 448`:

```javascript
for (let i = 0; i < W.sect.gear.length; i++) { const gid = W.sect.gear[i].id; if (gid) gearCounts[gid] = (gearCounts[gid] || 0) + 1; }
...
const gcount = gearCounts[r.outItemId || r.id] || 0;
s += ... - gcount * 0.15;
```

- Every procedural sword—whether a broken Tier-1 scrap sword or a Tier-4 Mithril masterpiece—shares `gear.id === 'ITEM_PROC_SWORD'` (`line 718`)! If `4` low-tier `ITEM_PROC_SWORD` items sit in `W.sect.gear`, `REC_PROC_SWORD` suffers `-0.60` penalty even when the sect just mined Tier-4 Mithril!
- **Second Penalty (`line 453`)**: `ECO.outputValue(W, REC_PROC_SWORD)` evaluates `ITEM_PROC_SWORD` as static `tier: 1` (`value ~ 16`) while `ECO.inputCost` sums raw ores (`~40`), yielding **`0` perceived craft margin**!
- **Fix**: Evaluate `REC_PROC_<TYPE>` expected tier/value in `ECO.pickOrder` using `ECO.previewProceduralCraft(W, r, null, head)` (`estTier`), and count `gcount` only for armory items with `gear.tier >= estTier`!

---

2. `[PROC-MAT-02]` Three Bugs in `slotOptions` & `ECO.resolveCraftMaterials` (`js/economy.js:95–155`)

1. **`MAT_V230_001..024` `materialMods` Are Completely Ignored (`economy.js:136–143`)**:
   - Patch `v2.3.0` added **24 T1–T7 Grand Dao crafting materials** (`Tamahagane Steel`, `Kuroi Chakra Metal`, `Khmer Cinnabar`, `Temporal Sand`, `Void Crystal`, etc.), storing `mat.materialMods = { tierCap, elementBias, statBias, qShift }` on each material definition.
   - **Bug**: `ECO.resolveCraftMaterials` (`lines 136–143`) reads **only** `co.tierCap / co.elementBias / co.statBias` on the recipe's `slotOptions` entry, **never merging `matById[co.matId].materialMods`**, AND the 17 legacy `REC_PROC_SWORD..CHARM` recipes never had `MAT_V230_*` added to their `slotOptions`!
   - **Fix**: Merge `Object.assign({}, matById[co.matId]?.materialMods, co)` inside `resolveCraftMaterials`, and dynamically inject compatible `MAT_V230_*` (T1–T4) options into `REC_PROC_* slotOptions` by material category (`ORE` into core slot, `BEAST_PART`/`HERB`/`FORMATION_PART` into secondary slot)!
2. **Crafter Stage vs Forging Skill Mismatch (`economy.js:103`)**:

- `maxStage = clamp(effectiveTechStage(npc) + 1, 1, 4)` blocks a dedicated Stage-2 Blacksmith (`FORGING = 82`, `forge.level = 3`) from using Tier-4 Mithril Ore (`mTier = 4 > maxStage 3`, penalized by `-14` score!). Use `Math.max(effectiveTechStage(npc) + 1, Math.floor((npc.h.skills.FORGING || 0) / 22) + 1)` clamped to `1..4`.

3. **Ignores `ECO.reserveFloor` (`economy.js:119`)**:
   - Procedural forging drains the very last herbs/beast cores down to `0` instead of preserving `ECO.reserveFloor(W, o.matId)` for emergency pills!

---

3. `[PROC-RES-03]` Critical Set-Hijacking Bug: `gear.resonanceSetId = matched[0]` Forces Every Procedural Set Onto `XIAN`/`BODY` Sets (`js/economy.js:729–740` vs `martial.js:354`)!

- **Exact Code (`js/economy.js:729–740`)**:

```JavaScript
if (tier >= 3 && JD.MARTIAL && Array.isArray(JD.MARTIAL.RESONANCE_SETS)) {
  const matched = [];
  for (const rs of JD.MARTIAL.RESONANCE_SETS) { ... matched.push(rs.id); }
  if (matched.length) {
    gear.resonanceSetIds = matched;
    gear.resonanceSetId = matched[0]; // <-- BUG! ALWAYS PICKS THE FIRST SET IN ARRAY ORDER!
  }
}
```

- **Exact Math in `M.evaluateResonanceSet` (`js/martial.js:354–355`)**:

```JavaScript
const explicitPieces = matched.reduce((count, sl) => count + (usable[sl].resonanceSetId === rs.id ? 1 : 0), 0);
const score = matched.length * 100 + explicitPieces * 60 + (subMatch ? 35 : 0) + (pathMatch ? 12 : 0);
```

- **Why This Breaks 90% of Subpaths**:
  - Dozens of Resonance Sets share the same `slotTypes` + `primaryEl` (e.g., `FIRE SWORD` + `ROBE` + `RING`).

    - Because `economy.js:738` stamps `gear.resonanceSetId = matched[0]` (the very first matching set in `RESONANCE_SETS`, always an early `XIAN`/`BODY` baseline set), that set receives **`explicitPieces * 60 = +180` points**, **crushing the wearer's actual subpath & path set (`+35 + 12 = +47` points)**!
- **Exact Fix**: On procedural gear, keep `gear.resonanceSetIds = matched` (which `itemMatchesResonanceSet` line 327 already checks!), and **do NOT stamp `gear.resonanceSetId = matched[0]`** unless `spec.npc` has a matching subpath set in `matched` (or leave `gear.resonanceSetId = null` so `evaluateResonanceSet` picks the *wearer's* matching `subMatch + pathMatch` set naturally!).

---

4. `[PROC-AFF-04]` Affix Pool Mismatches & Unrestricted `ASTRAL` / `BLOOD_DEMON` Rolls on Mundane Tier-1 Iron (`economy.js:600–680`)

1. **Dead Off-Type Offensive Affixes & Absurd Generated Names (`economy.js:600–614, 654–662`)**:
    - Pure physical weapons (`BLADE`, `FIST`, `CLAW`, `SPEAR`, `BOW`) roll `profound_matk` (สะท้อนปราณ — dead stat on `BODY` / `SHADOW_SAMURAI`!), while pure magical conduits (`FOCUS`, `ZITHER`, `BRUSH`, `ROSARY`, `ASTROLABE`) roll `ferocious_atk` (ขย้ำพยัคฆ์ — generating names like ลูกประคำบัวแก้วโพธิญาณ·ขย้ำพยัคฆ์ *"Tiger-Mauling Bodhi Rosary"* with dead physical `atk`!).
    - **Fix**: Filter `profound_matk` out of pure-physical weapon types (`BLADE`, `FIST`, `CLAW`, `BOW`), filter `ferocious_atk` out of pure-`matk` conduits (`FOCUS`, `ZITHER`, `BRUSH`, `ROSARY`, `ASTROLABE`), and allow `gale_eva` / `stride_move` on agile weapons (`FAN`, `BOW`, `NEEDLE`, `CLAW`, `WHIP`).
2. **Mundane Tier-1 Iron Uniformly Rolls `ASTRAL` & `BLOOD_DEMON` (`18.2%` Rate, `economy.js:642`)**:
    - Without material biases, `ALL_ELEMENTS` (`11` elements) gives **18.2% of Tier-1 Black-Iron gear** locked Special Elements (`ASTRAL` or `BLOOD_DEMON`), penalizing orthodox novices who lack `specialAccess` and breaking lore when righteous sects forge Demonic Blood gear from plain iron!
    - **Fix**: Require `tier >= 3` or an explicit `ASTRAL` / `BLOOD_DEMON` `elementBias` material to roll `ASTRAL` or `BLOOD_DEMON`, weighting `'NEUTRAL'` (`25%`) higher on un-biased Tier 1–2 mundane iron/leather!
3. **16-Weapon Un-Typed Drop Dilution (`economy.js:627`)**:
    - When merchants (`external_market.js`) or ruins (`explore.js`) call `rollProceduralGear(W, { slot: 'WEAPON' })` with no `gearType`, uniform picking across `16` weapon classes yields `68%+` exotic weapons (`ABACUS`, `ASTROLABE`, `ROSARY`, `NEEDLE`) unusable by early 1-path sects! Weight

un-specified merchant/drop weapon rolls 45% toward unlocked sect/regional path weapons + 55% universal Jianghu pool.

---

5. [PROC-DUR-05] Severe Durability Stat-Penalty Curve & dur-above-new-max Repair Dead-Zone (stats.js:368, 598 & economy.js:814–818)!

1. **Freshly Repaired Procedural Gear Permanently Loses Up to -70% Stats (stats.js:368 & 598)!**:
    - Look at js/stats.js:368 (and 598):

```javascript
let df = it.dur == null ? 1 : clamp(it.dur / 100, 0.3, 1);
```

    - **Two Harsh Mathematical Flaws**:
        - **Flaw A**: At dur = 60 / 100, an unbroken sword **already loses -40% of all stats**! Normal wear above 50% of maxDur should retain 100% effectiveness (df = 1.0), tapering gently only when badly chipped (< 50%).
        - **Flaw B**: Repairing a procedural weapon reduces maxDur (100 -> 80 -> 60 -> 40). At **dur = 40 / maxDur = 40 (100% freshly repaired!)**, it.dur / 100 evaluates to **0.40 (-60% of all stats permanently lost even right off the repair anvil!)**, making repaired procedural gear worse than a Tier-1 stick after just 2–3 repairs!
    - **Fix**: Compute condition from curRatio = it.dur / Math.max(20, it.maxDur || 100) (1.0 when curRatio >= 0.50, 0.65 + 0.70 * curRatio when 0 < curRatio < 0.50) multiplied by a gentle wear-aging factor 0.85 + 0.15 * ((it.maxDur || 100) / 100) (max -15% at maxDur = 20, matching martial.js:602 0.75 + 0.25*(maxD/100)!).
2. **dur-above-new-max Repair Dead-Zone (economy.js:814–818) & Blind Iron Salvage (line 867)**:
    - In SIM.hourly, if an NPC's gear is at dur = 42 / maxDur = 60 and only ITEM_REPAIR_STONE_1 (-25 -> newMaxDur = 35) is in stock, curDur (42) >= newMaxDur (35) fails every hour without repairing, then once dur drops to 34, repairing cuts maxDur to 35! Trigger autonomous repair only when curDur <= Math.min(curMax * 0.35, curMax - stone.maxDurReduction - 5).
    - In ECO.retireGear (line 867), check gear.origin && Array.isArray(gear.origin.materials) first to return 1 Tier-appropriate

material matching what the item was actually made of (e.g. wood/silk/jade/ore up to Tier 3) instead of turning silk robes and wooden zithers into `MAT_ORE_COLD_STEEL_2`!

---

6. Implementation & Verification Checklist for GPT / Gemini (`05F`)

- [ ] **Sync All `23` `REC_PROC_*` & `ITEM_PROC_*` Templates**: Ensure all `23` types in `ALL_GEAR_TYPES` exist in **both** `JD.ITEMS.recipes` and `ECO.recipeById` with `proceduralOut: true`, `slotOptions`, and `ECO.recipesFor` visibility; fix `ECO.pickOrder` `estTier` valuation and `gcount` tier-filtering.
- [ ] **Upgrade `resolveCraftMaterials` (`economy.js:95–155`)**: Merge `matById[matId].materialMods`, integrate `MAT_V230_001..024` (T1–T4) into `slotOptions`, respect `FORGING` skill on `maxStage`, and honor `ECO.reserveFloor`.
- [ ] **Fix Procedural Resonance Hijacking (`economy.js:738`)**: Stop hard-stamping `gear.resonanceSetId = matched[0]` on procedural gear so `evaluateResonanceSet` matches the wearer's own subpath set via `gear.resonanceSetIds`.
- [ ] **Refine Affix & Element Pools (`economy.js:600–665`)**: Gate `atk`/`matk` affixes by weapon role, gate `ASTRAL`/`BLOOD_DEMON` on T3+ or material bias, and weight un-typed merchant/drop `gearType` rolls sensibly.
- [ ] **Fix Durability Curve & Salvage (`stats.js:368, 598`, `economy.js:814, 867`)**: Replace raw `it.dur / 100` linear stat collapse with `curRatio` + gentle `maxDur` aging, align auto-repair thresholds, and salvage material matching `gear.origin.materials`.